

Frontend Intern Assignment

AI-Powered Finance Dashboard (Personal Finance)

Objective

Build a **production-quality Personal Finance Dashboard** based on the provided design.

The goal is to evaluate:

- Real-world frontend engineering skills
- Ability to work with **complex UI systems (fintech-grade)**
- Understanding of **performance, analytics, SEO, and scalable architecture**

Assignment Overview

You are required to implement the **Personal Finance – Wealth Curator Dashboard** (shared design) using **React(react-native-web)** or **React Native**(Plus points).

The dashboard includes:

- Net worth and financial summaries
- AI-driven insights and alerts
- Transactions and spending analytics
- Portfolio and budget intelligence

Example UI Link:

<https://drive.google.com/file/d/10yD14gh3frLxjJK75QbUEN5-Ee-B-b9y/view?usp=sharing>

Core Requirements

1. UI Implementation (High Priority)

- Implement the dashboard **pixel-accurately** from the design
- Use **component-based architecture**
- Follow consistent spacing, typography, and layout system

Key sections to build:

- Header (search, profile, navigation)
- Summary cards (Net Worth, Spending, Savings)
- AI Insights / Strategy cards
- Alerts section
- Transactions table/list
- Spending breakdown (charts optional)

2. Custom Hooks (Mandatory)

Create reusable hooks:

- `useFetch` – API/data handling
- `useAnalytics` – event tracking
- `useDebounce` – for search/filter
- `useLocalStorage` – persist user preferences

Focus on:

- Clean abstraction
- Reusability
- Separation of concerns

3. Data Handling

- Use mock JSON or dummy API
- Implement:
 - Loading states
 - Error handling
 - Empty states

4. Google Analytics / Tagging

Integrate **Google Analytics (GA4)** or **Google Tag Manager**

Track events such as:

- Page load
- Search usage
- Filter clicks
- “Execute Strategy” / CTA clicks

5. Performance Optimization

- Lazy loading (React.lazy / Suspense)
- Code splitting
- Memoization (React.memo, useMemo, useCallback)
- Avoid unnecessary re-renders

6. SEO & Accessibility (Web)

- Semantic HTML (header, main, section)
- Meta tags (title, description)
- Basic Open Graph tags
- Accessibility:
 - Alt text
 - Labels
 - Keyboard navigation

7. AI Insights (Simulated Logic)

Use mock logic to generate insights like:

- “Your tech exposure increased by 14%”
- “You can save \$180 annually by removing duplicate subscriptions”

Insights should:

- Be dynamic (based on data)
- Match the tone of the UI (premium/financial)

Bonus (Strong Differentiator)

- React Native Web support (same components reusable) - <https://necolas.github.io/react-native-web/>
- Dark mode implementation
- Chart integration (Recharts / Chart.js)
- Design system implementation (tokens for spacing, colors)
- Table optimizations (virtualization for large data)

Deliverables

- GitHub repository
- Live deployed link (Vercel / Netlify)
- README including:
 - Architecture decisions
 - Custom hooks explanation
 - Performance optimizations
 - SEO techniques used
 - Trade-offs